

Modelo de Clasificación Binaria para la Predicción de Abandono de Clientes a partir de Datos de Suscripción: Dataset Telco Customer Churn

Nátaly Cárdenas^{1*} and Laura Rivera¹

^{1*}Departamento de Matemáticas, Física y Ciencia de Datos,
Universidad del Norte, Barranquilla, Colombia.

*Corresponding author(s). E-mail(s): nizaquita@uninorte.edu.co;
Contributing authors: srivaler@uninorte.edu.co;

Abstract

El abandono de clientes (*churn*) en telecomunicaciones genera pérdidas recurrentes de ingresos, y detectarlo a tiempo permite focalizar acciones de retención. Este trabajo aborda la clasificación binaria del abandono sobre el conjunto público Telco Customer Churn. Se diseñó un protocolo reproducible con preprocesamiento sin fuga de información y validación cruzada anidada (5 particiones externas y 3 internas), en el que se compararon siete familias de modelos (KNN, Naive Bayes gaussiano, regresión logística, árbol de decisión, Random Forest, XGBoost y SVM), cuatro estrategias de balanceo y cuatro optimizadores de hiperparámetros (Grid, Random, TPE y algoritmo genético), para un total de 112 configuraciones. Random Forest con SMOTE obtuvo el mayor F1-score de la clase de abandono (0.638 ± 0.023), seguido de cerca por XGBoost (0.638) y la regresión logística (0.634), con un AUC-ROC de 0.844 en el test histórico. Las diferencias entre los tres mejores modelos fueron pequeñas y no detectables estadísticamente, mientras que SMOTE mejoró el F1-score en seis de las siete familias frente a no balancear, y Random Forest fue el modelo de mayor costo de búsqueda. SHAP identificó el tipo de contrato, el servicio de internet y la antigüedad como las variables más influyentes. Los resultados de esta etapa corresponden a la comparación de modelos de referencia y sirven como base para la posterior formulación del modelo propuesto.

Keywords: Predicción de abandono de clientes, Validación cruzada anidada, Desbalance de clases, Optimización de hiperparámetros, F1-score

1 Introducción

La retención de clientes es una prioridad para las empresas de telecomunicaciones. Identificar qué clientes están en riesgo de abandonar el servicio permite dirigir ofertas y acciones de retención de forma selectiva, pero exige un modelo que detecte bien la clase minoritaria de abandono sin generar un volumen excesivo de falsas alertas. El problema es difícil por dos razones: la clase de interés es minoritaria y las variables disponibles describen el estado del cliente en un corte, lo que limita la lectura prospectiva de cualquier métrica.

La literatura y la práctica emplean sobre todo modelos clásicos de aprendizaje supervisado, como la regresión logística [1], los ensambles de árboles como Random Forest [2] y los métodos de *boosting* como XGBoost [3], junto con vecinos cercanos [4], clasificadores bayesianos ingenuos [5], árboles de decisión [6] y máquinas de vectores de soporte [7]. Para el desbalance se recurre a técnicas de remuestreo sintético como SMOTE [8] y ADASYN [9], o a ponderaciones de clase. Sin embargo, en una comparación entre modelos resulta difícil separar el efecto de la familia, el del balanceo y el del método de búsqueda de hiperparámetros, y seleccionar modelos con el mismo conjunto en el que se reporta su desempeño infla las estimaciones [10].

La contribución de este trabajo es un diseño experimental controlado y reproducible: siete familias de modelos, cuatro estrategias de balanceo y cuatro optimizadores de hiperparámetros (112 configuraciones) se evalúan con los mismos folds, con preprocesamiento y balanceo ajustados únicamente dentro de cada partición de entrenamiento, y con selección de hiperparámetros por validación cruzada anidada. Se complementa con calibración de probabilidades, comparación estadística explícita sobre sus alcances e interpretabilidad con SHAP [11] y LIME [12]. En esta etapa mostramos los resultados de la comparación entre los modelos de referencia: Random Forest con SMOTE y XGBoost con SMOTE alcanzan los mayores F1-score medios (0.638), sin diferencias estadísticamente detectables frente a los principales modelos comparados. Además, el balanceo aporta más que el cambio de optimizador cuando el presupuesto de búsqueda es limitado.

2 Metodología

2.1 Descripción del problema y del conjunto de datos

Descripción general. Se utilizó el conjunto público Telco Customer Churn de IBM [13], con 7,043 clientes y 21 columnas: un identificador, 19 variables predictoras (3 numéricas y 16 categóricas, incluida `SeniorCitizen`, almacenada como entero) y la variable objetivo `Churn`. La variable objetivo es binaria (Yes = el cliente abandona, No = permanece) y presenta desbalance: en el conjunto de entrenamiento, 4,139 clientes permanecen (73.46%) y 1,495 abandonan (26.54%).

ETL. *Extract:* los datos se cargaron completos, sin muestreo, desde el archivo CSV original. *Transform:* la columna `TotalCharges` se leyó como texto porque 8 registros del entrenamiento (3 en el test) contenían un espacio en blanco en lugar de un importe; se convirtió a numérico y se reemplazó con 0, tras verificar con una aserción que todos esos registros tienen `tenure` = 0 (clientes recién dados de alta) e

inferir que ambas variables están relacionadas (sin antigüedad no hay cargo acumulado). Se colapsaron las categorías redundantes “No internet service” y “No phone service” en “No”, pues el EDA no encontró discrepancias relevantes y esta transformación reduce categorías redundantes mediante *feature engineering*. Se creó la variable **AvgChargePerMonth** (cargo acumulado dividido por antigüedad, con **MonthlyCharges** como respaldo cuando **tenure** = 0). Las variables categóricas se codificaron con One-Hot (primera categoría eliminada) y las cuatro numéricas (**tenure**, **MonthlyCharges**, **TotalCharges** y **AvgChargePerMonth**) se estandarizaron, lo que produce 24 columnas. No se eliminaron valores atípicos. *Load*: el conjunto se particionó en 80/20 de forma estratificada con semilla aleatoria fija (**seed**=42), lo que da 5,634 clientes de desarrollo y 1,409 de test; la selección de modelos se hizo con validación cruzada anidada sobre los 5,634 clientes de desarrollo, y el test se usó solo como evidencia suplementaria. El escalado, la codificación y el balanceo aprenden sus parámetros únicamente con los datos de entrenamiento de cada partición.

EDA. Todo el análisis exploratorio se realizó solo sobre el conjunto de entrenamiento. La estadística descriptiva muestra **tenure** entre 0 y 72 meses (media 32.5), **MonthlyCharges** entre 18.4 y 118.75 (media 64.9) y **TotalCharges** con asimetría positiva de 0.95. El criterio IQR no marcó ningún valor atípico en las tres variables numéricas. Los únicos valores faltantes son los 8 de **TotalCharges** (0.14%). Los clientes que abandonan tienen menor antigüedad y menores cargos acumulados, pero mayores cargos mensuales; la asociación numérica marginal más fuerte con el abandono es la de **tenure** (correlación punto-biserial $r_{pb} = -0.3456$). Entre las variables categóricas, la asociación más fuerte es la de **Contract** (V de Cramér = 0.4111 [14]), seguida de **OnlineSecurity** (0.3510) y **TechSupport** (0.3449), mientras que **gender** (0.0022) y **PhoneService** (0.0179) no muestran evidencia de asociación tras la corrección de Holm [15] sobre 19 contrastes. Las mayores tasas de abandono se observan con el cheque electrónico (45.74%), el contrato mensual (42.75%), la fibra óptica (42.09%), la ausencia de seguridad en línea (41.94%) y la ausencia de soporte técnico (41.75%), como resume la Figura 2. Además, ninguna de las tres variables numéricas superó la prueba de normalidad de D’Agostino–Pearson (con ajuste de Holm, valor p prácticamente nulo en todas), por lo que la correlación entre predictores se calculó con el coeficiente de Spearman, y **tenure** y **TotalCharges** resultaron fuertemente correlacionadas ($\rho = 0.8914$).

Como consecuencia directa de estos hallazgos se tomaron las siguientes decisiones de preprocesamiento: (i) el desbalance de 73.5/26.5 llevó a evaluar la clase de abandono con métricas de clase positiva (F1, Recall, Precision) y a comparar cuatro estrategias de balanceo; (ii) la fuerte correlación entre **tenure** y **TotalCharges** llevó a conservar ambas y a añadir **AvgChargePerMonth** en lugar de eliminar una de ellas; (iii) la información duplicada de las categorías de ausencia de servicio motivó colapsarlas; (iv) los 8 faltantes de **TotalCharges**, todos con **tenure** = 0 (clientes recién dados de alta, sin cargo acumulado), justificaron reemplazarlos por 0, decisión contrastada con la alternativa de mediana más indicador de imputación, que dio prácticamente el mismo AUC-ROC en validación cruzada (0.8459 y 0.8460); (v) la ausencia de valores atípicos según el IQR llevó a no recortar ni eliminar observaciones; y (vi) **SeniorCitizen**, por ser una variable binaria, se trató como categórica y no se estandarizó. **gender** y

Marco experimental del estudio

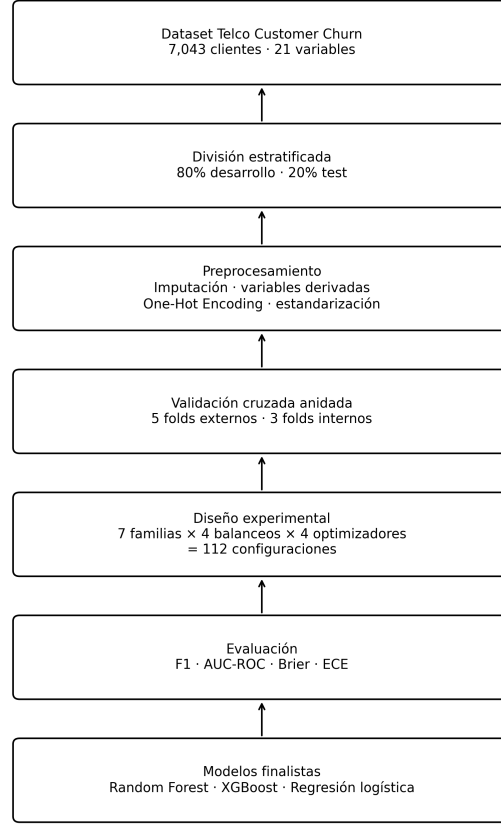


Fig. 1 Marco experimental del estudio. El protocolo comprende la división estratificada de los datos, el preprocesamiento, la validación cruzada anidada y la evaluación de siete familias de modelos bajo cuatro estrategias de balanceo y cuatro optimizadores de hiperparámetros.

PhoneService se conservaron pese a su falta de asociación bivariada, y una ablación por validación cruzada mostró que quitarlas no cambia el AUC-ROC de forma medible (diferencias de $+0.0001$ a $+0.0004$, del orden de la variación entre folds).

2.2 Modelos de referencia (baselines)

Se compararon siete familias de modelos, de modo que ninguna se descartó por su desempeño: **regresión logística** [1], como referencia lineal regularizada; **K vecinos más cercanos (KNN)** [4]; **Naive Bayes gaussiano** [5]; **árbol de decisión** [6]; **Random Forest** [2], por su robustez ante datos tabulares mixtos; **XGBoost** [3], referencia de *boosting* de árboles; y **SVM con kernel RBF** [7]. Las implementaciones

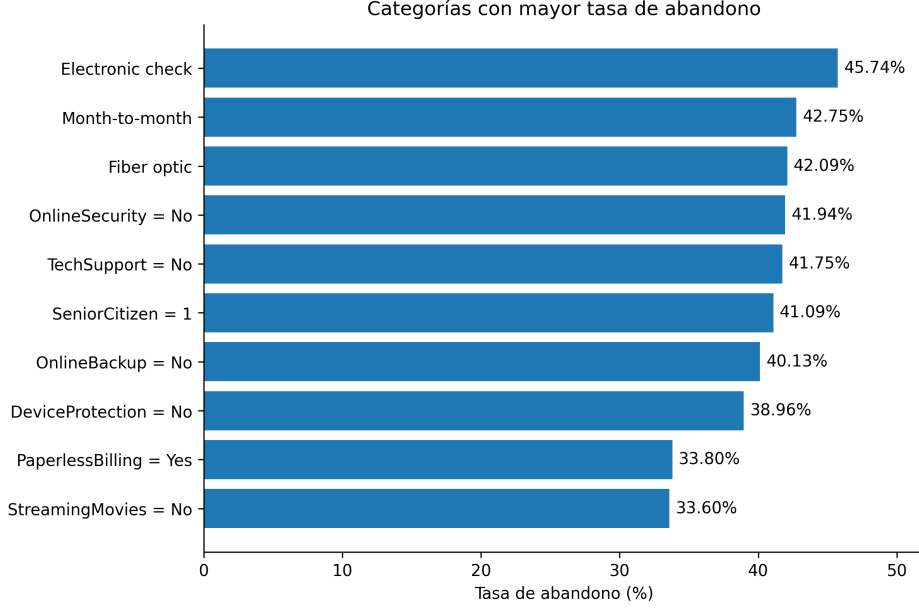


Fig. 2 Tasa de abandono por categoría en el conjunto de entrenamiento (EDA). El contrato mensual, el cheque electrónico, la fibra óptica y la ausencia de seguridad en línea o soporte técnico concentran las mayores tasas de abandono, sobre una prevalencia global de 26.54%.

proviene de scikit-learn [16] y XGBoost. A continuación se resume la formulación de cada una.

Regresión logística. Modela $P(y = 1 | x) = \sigma(w^\top x + b)$ con $\sigma(z) = 1/(1 + e^{-z})$ y minimiza una pérdida logarítmica regularizada, donde C es el inverso de la regularización y Ω es la norma L_1 o L_2 [1]:

$$\min_{w,b} \Omega(w) + C \sum_{i=1}^n \log(1 + e^{-y_i(w^\top x_i + b)}), \quad y_i \in \{-1, +1\}. \quad (1)$$

KNN. Clasifica x por el voto de sus k vecinos más cercanos $\mathcal{N}_k(x)$ según la distancia de Minkowski de orden p , con pesos $w_i = 1$ (uniforme) o $w_i = 1/d(x, x_i)$ (por distancia) [4]:

$$\hat{y}(x) = \arg \max_c \sum_{i \in \mathcal{N}_k(x)} w_i \mathbb{I}[y_i = c], \quad d_p(x, x') = \left(\sum_j |x_j - x'_j|^p \right)^{1/p}. \quad (2)$$

Naive Bayes gaussiano. Supone independencia condicional de las variables dada la clase, con densidades gaussianas; `var_smoothing` añade un término ϵ a las varianzas [5]:

$$\hat{y}(x) = \arg \max_c \pi_c \prod_{j=1}^p \mathcal{N}(x_j; \mu_{cj}, \sigma_{cj}^2 + \epsilon). \quad (3)$$

Árbol de decisión. En cada nodo t elige la partición que más reduce una impureza I , que puede ser Gini o entropía, con p_c la proporción de la clase c en el nodo [6]:

$$I_{\text{Gini}}(t) = 1 - \sum_c p_c^2, \quad I_{\text{ent}}(t) = - \sum_c p_c \log p_c. \quad (4)$$

Random Forest. Promedia (o vota) T árboles h_t entrenados con muestras bootstrap $\mathcal{D}_t$ y un subconjunto aleatorio de variables en cada nodo [2]:

$$\hat{y}(x) = \text{mayoría}\{h_t(x; \mathcal{D}_t)\}_{t=1}^T. \quad (5)$$

XGBoost. Construye un modelo aditivo $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$ que minimiza una pérdida regularizada, donde T_f es el número de hojas del árbol y w los pesos de sus hojas [3]:

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_t \Omega(f_t), \quad \Omega(f) = \gamma T_f + \frac{1}{2} \lambda \|w\|^2. \quad (6)$$

SVM con kernel RBF. Resuelve el problema de margen suave y, vía el truco del kernel, produce la función de decisión [7]:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.a.} \quad y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad f(x) = \text{sign} \left(\sum_i \alpha_i y_i k(x_i, x) + b \right), \quad (7)$$

con $k(x_i, x) = \exp(-\gamma \|x_i - x\|^2)$.

Los hiperparámetros se ajustaron con cuatro optimizadores sobre los espacios de la Tabla 1, fijados antes de observar resultados. Grid enumera exactamente 24 combinaciones por familia; los demás optimizadores usan el mismo soporte categórico o los mismos límites continuos y consumen el mismo presupuesto de 24 evaluaciones únicas.

Table 1 Espacios de búsqueda de hiperparámetros por familia (Grid: 24 candidatos).

Familia	Espacio de búsqueda
KNN	$k \in \{3, 5, 11, 21, 41, 81\}$; pesos $\in \{\text{uniforme, distancia}\}$; $p \in \{1, 2\}$
Naive Bayes	<code>var_smoothing</code> : 24 valores log en $[10^{-12}, 10^{-1}]$
Reg. logística	C : 12 valores log en $[10^{-3}, 10^3]$; penalización $\in \{L_1, L_2\}$
Árbol de decisión	profundidad $\in \{3, 6, 12, \text{None}\}$; hoja mín. $\in \{1, 5, 20\}$; criterio $\in \{\text{gini, entropía}\}$
Random Forest	$T \in \{100, 200, 300\}$; profundidad $\in \{8, \text{None}\}$; hoja mín. $\in \{1, 5\}$; variables $\in \{\sqrt{p}, 0.7p\}$
XGBoost	$T \in \{100, 300\}$; profundidad $\in \{2, 4, 6\}$; $\eta \in \{0.03, 0.1\}$; submuestreo $\in \{0.8, 1\}$
SVM (RBF)	$C \in \{0.01, 0.1, 1, 10, 100, 1000\}$; $\gamma \in \{0.001, 0.01, 0.1, 1\}$

2.3 Marco experimental

El marco experimental de esta etapa es un protocolo controlado para comparar las siete familias de modelos, las cuatro estrategias de balanceo y los cuatro optimizadores

de hiperparámetros bajo las mismas particiones y métricas. Esta sección documenta únicamente los modelos de referencia; la formulación y evaluación del modelo propuesto se desarrollarán posteriormente. Cada ajuste construye un *pipeline* nuevo con el orden: datos originales $\rightarrow$ reglas deterministas $\rightarrow$ escalado y codificación $\rightarrow$ balanceo del entrenamiento $\rightarrow$ clasificador, de modo que en validación y prueba solo se transforma y predice, sin remuestrear.

En KNN, árbol de decisión y Random Forest, los cuatro optimizadores agotan el mismo espacio discreto de 24 candidatos; cuando existe empate a precisión numérica, se conserva el identificador de configuración definido por el protocolo. Por ello, la etiqueta TPE que aparece en la configuración seleccionada de estas familias no debe interpretarse como una ventaja estadística del optimizador.

Algorithm 1 Validación cruzada anidada con balanceo y optimización de hiperparámetros

Require: Datos de desarrollo D , configuración (familia, balanceo, optimizador), 5 folds externos, 3 internos, presupuesto $B = 24$

Ensure: Predicciones fuera de muestra y métricas por fold externo

```

1: for cada fold externo  $k = 1, \dots, 5$  do
2:   Reservar el fold  $k$  como evaluación, sin usar sus etiquetas para elegir parámetros
3:   for cada candidato  $\theta$  propuesto por el optimizador (hasta  $B$  evaluaciones únicas) do
4:     for cada fold interno  $j = 1, \dots, 3$  do
5:       Ajustar escalado, codificación, balanceo y clasificador solo con el
       entrenamiento interno
6:       Calcular F1 positivo en el fold interno de validación
7:     end for
8:     Devolver al optimizador la media de F1 interno de  $\theta$ 
9:   end for
10:  Elegir  $\theta^*$  con mayor media interna y reajustar el pipeline en todo el entre-
  namiento externo
11:  Predecir el fold  $k$  y guardar etiquetas, scores y tiempos
12: end for
13: return métricas externas (media  $\pm$  desviación estándar de 5 folds)

```

2.4 Configuración experimental

Se utilizó **F1 positivo** como métrica principal, complementada con Precision, Recall, Accuracy, AUC-ROC, Brier y ECE. Para la clase de abandono,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2TP}{2TP + FP + FN}.$$

El F1 se definió antes de ejecutar la comparación y se utilizó como único objetivo de los optimizadores porque resume el equilibrio entre detectar clientes que abandonan

Algorithm 2 Reajuste final de los tres modelos finalistas y evaluación histórica

Require: Conjunto de desarrollo D , tres configuraciones congeladas, presupuesto $B = 24$, tres folds internos

Ensure: Modelos finales, predicciones y métricas sobre el test histórico

- 1: **for** cada configuración finalista m **do**
 - 2: Ejecutar la búsqueda definida para m con $B = 24$ evaluaciones y tres folds internos
 - 3: Seleccionar θ_m^* por la mayor media de F1 positivo interno
 - 4: Reajustar el preprocesamiento, SMOTE y clasificador sobre todo D usando θ_m^*
 - 5: Congelar m antes de consultar el test histórico
 - 6: Predecir el test sin remuestreo y guardar etiquetas, scores y probabilidades
 - 7: Calcular Precision, Recall, F1, Accuracy, AUC, Brier y ECE
 - 8: **end for**
 - 9: Construir 2,000 remuestreos pareados y estratificados del test para IC 95%
 - 10: Comparar AUC con DeLong y corregir los tres contrastes mediante Holm
-

y evitar falsas alertas. No se dispone de una función de costes económicos que permita afirmar que maximizar F1 equivale a maximizar rentabilidad. Por ello, Precision, Recall y AUC-ROC se conservaron como métricas complementarias y no se ajustaron umbrales sobre el test.

El diseño factorial cubre 7 familias $\times$ 4 balanceos $\times$ 4 optimizadores = 112 configuraciones, cada una con 3 realizaciones algorítmicas derivadas de la semilla raíz 42 y 5 folds externos, para 1,680 unidades externas. Las realizaciones reutilizan las mismas particiones, pero modifican las semillas asociadas al optimizador, al modelo y al remuestreo. Esta variación mide sensibilidad algorítmica con particiones fijas; no representa tres muestras independientes de clientes.

La validación externa utilizó cinco particiones estratificadas y la selección de hiperparámetros tres particiones internas. El presupuesto fue de 24 evaluaciones únicas por optimizador y configuración. Para los tres modelos finalistas –Random Forest–SMOTE–TPE, XGBoost–SMOTE–Random y regresión logística–SMOTE–Grid– se realizó posteriormente una selección adicional sobre todo el conjunto de desarrollo, con tres folds internos y 24 evaluaciones. Esto implica $3 \times (24 \times 3 + 1) = 219$ ajustes adicionales: 216 ajustes internos y tres reajustes finales. Estos 219 ajustes se registraron por separado y no amplían las 112 configuraciones ni alteran sus resultados externos originales.

En el test histórico se calcularon intervalos de confianza percentiles del 95% mediante 2,000 remuestreos pareados y estratificados, utilizando los mismos índices para los tres modelos. Las diferencias de AUC se compararon mediante DeLong y se aplicó la corrección de Holm a los tres pares. No se ajustó el umbral de decisión usando el test. Las probabilidades se evaluaron además mediante calibración sigmoide de Platt, con Brier y ECE calculados en diez intervalos uniformes.

El entorno computacional utilizó Python 3.12 y las bibliotecas scikit-learn [16], imbalanced-learn [17], XGBoost, Optuna [18], DEAP [19], SHAP y LIME. Los experimentos se ejecutaron en un equipo con CPU Intel Core Ultra 5 125H (14 núcleos y 18 hilos lógicos), aproximadamente 8 GB de RAM, sin GPU CUDA disponible y sistema

operativo Windows. El paralelismo se limitó a un nivel (hasta dos folds internos, hilos numéricos acotados y ejecución serial si había menos de 1.5 GiB libres), de modo que no se atribuye ninguna aceleración a GPU. Un *benchmark* auxiliar con tres repeticiones por familia comparó solicitar uno o dos trabajadores: la política de memoria activó un solo trabajador efectivo en ambos casos, por lo que esos tiempos no demuestran aceleración por paralelismo. Los tiempos de búsqueda, reajuste e inferencia se registraron por separado.

Las tablas reportan media $\pm$ desviación estándar muestral de los cinco folds externos de la realización principal. Los resultados internos, los folds externos, las predicciones, los scores, las probabilidades, los hiperparámetros, las semillas y los tiempos se conservaron por separado para mantener trazabilidad entre selección y evaluación. *Fase de confirmación sobre los finalistas.* Las tres familias de mayor F1 medio externo se congelaron, con su mejor optimizador y balanceo, antes de consultar el test histórico. Para cada una se repitió la selección de hiperparámetros con tres folds internos sobre todo el desarrollo y 24 evaluaciones, seguida de un ajuste final: 219 ajustes adicionales ($3 \times [24 \times 3 + 1]$) que no amplían las 112 configuraciones ni sus 1,680 resultados externos. Sobre el test se calcularon intervalos percentiles del 95% con 2,000 remuestreos pareados y estratificados de clientes (los mismos índices para todos los modelos) y la prueba de DeLong para las diferencias de AUC entre los tres modelos fijos, con corrección de Holm sobre la familia predefinida de tres pares.

3 Resultados

La Tabla 2 resume la mejor configuración de cada familia. Random Forest–SMOTE–TPE y XGBoost–SMOTE–Random alcanzaron los mayores F1 medios externos, con 0.638 ± 0.023 y 0.638 ± 0.021 , respectivamente, mientras que la regresión logística alcanzó 0.634 ± 0.019 . La diferencia entre Random Forest y XGBoost es de apenas 0.0004 y queda por debajo de la variabilidad observada entre los folds. Random Forest no debe describirse como ganador exclusivo: su resultado se interpreta como parte de la línea base comparativa y no como evidencia de superioridad estadísticamente demostrada.

El balanceo fue el factor más influyente: SMOTE mejoró el F1 medio frente a no balancear en seis de las siete familias, con ganancias descriptivas desde 0.0210 en KNN hasta 0.0644 en Random Forest, y no aportó una mejora en Naive Bayes. La Tabla 3 muestra que el principal costo computacional de Random Forest se concentra en la búsqueda de hiperparámetros, mientras que el reajuste y la inferencia son breves.

Costo computacional. La Tabla 3 reúne los tiempos de la configuración seleccionada de cada familia. La búsqueda de Random Forest (unos 96 s) equivale a cerca de seis veces la de la regresión logística (16 s) y cuatro veces la de XGBoost (24 s); en cambio, su reajuste (cerca de 1 s) y su inferencia (menos de 0.1 s por bloque externo) siguen siendo bajos, de modo que el costo recae en la selección de hiperparámetros y no en el uso del modelo. SVM presenta la inferencia más lenta (0.48 s), seguida de KNN (0.23 s).

Comparación estadística. Se aplicó la secuencia Friedman $\rightarrow$ Nemenyi $\rightarrow$ diferencia crítica sobre las 112 configuraciones y los 5 folds externos [20, 21]. Friedman

Table 2 Mejor configuración de cada familia (realización principal, media $\pm$ desviación estándar de los 5 folds externos). En KNN, árbol de decisión y Random Forest los cuatro optimizadores empatan; se muestra TPE por desempate de identificador.

Familia	Balanceo-Optimizador	F1	Recall	Precision	Accuracy	AUC-ROC
Regresión logística	SMOTE-Grid	0.634 ± 0.019	0.781 ± 0.018	0.534 ± 0.027	0.760 ± 0.019	0.844 ± 0.009
KNN	SMOTE-TPE	0.613 ± 0.012	0.823 ± 0.019	0.488 ± 0.011	0.724 ± 0.010	0.831 ± 0.011
Naive Bayes	Sin balanceo-Random	0.620 ± 0.018	0.774 ± 0.020	0.517 ± 0.023	0.748 ± 0.017	0.830 ± 0.009
Árbol de decisión	SMOTE-TPE	0.612 ± 0.009	0.730 ± 0.026	0.527 ± 0.014	0.754 ± 0.010	0.824 ± 0.010
Random Forest	SMOTE-TPE	0.638 ± 0.023	0.735 ± 0.023	0.565 ± 0.027	0.779 ± 0.017	0.844 ± 0.010
XGBoost	SMOTE-Random	0.638 ± 0.021	0.745 ± 0.023	0.559 ± 0.029	0.775 ± 0.019	0.846 ± 0.011
SVM (RBF)	SMOTE-Genético	0.631 ± 0.019	0.779 ± 0.013	0.530 ± 0.025	0.758 ± 0.018	0.839 ± 0.010

Table 3 Métrica principal y costo computacional de la configuración seleccionada de cada familia. Búsqueda: media $\pm$ desviación estándar por búsqueda de 24 evaluaciones; los tiempos agrupan los cuatro balanceos de cada combinación familia-optimizador.

Modelo (balanceo-optimizador)	F1 externo	Búsqueda (s)	Reajuste (s)	Inferencia (s)
Random Forest (SMOTE-TPE)	0.638 ± 0.023	96.4 ± 20.0	1.06	0.069
XGBoost (SMOTE-Random)	0.638 ± 0.021	23.6 ± 6.1	0.23	0.039
Reg. logística (SMOTE-Grid)	0.634 ± 0.019	16.4 ± 3.4	0.18	0.037
SVM RBF (SMOTE-Genético)	0.631 ± 0.019	63.2 ± 34.9	0.73	0.482
Naive Bayes (sin balanceo-Random)	0.620 ± 0.018	10.9 ± 3.9	0.10	0.041
KNN (SMOTE-TPE)	0.613 ± 0.012	20.8 ± 12.5	0.11	0.231
Árbol de decisión (SMOTE-TPE)	0.612 ± 0.009	10.5 ± 0.9	0.10	0.034

produjo $\chi^2 = 422.29$ con 111 grados de libertad ($p = 7.7 \times 10^{-38}$) y una concordancia de Kendall $W = 0.76$; sin embargo, los folds de un solo conjunto de datos no son bloques independientes, por lo que estos resultados se leen solo como exploratorios. La diferencia crítica fue 89.4 rangos y solo 31 de los 6,216 pares quedaron bajo el umbral nominal 0.05, de modo que las mejores configuraciones no se separan entre sí. Entre los finalistas, la diferencia media de F1 entre Random Forest y XGBoost fue de apenas 0.0004 por fold.

Se congelaron las tres familias de mayor F1 medio externo, con su configuración de optimización definida por el protocolo, antes de consultar el test histórico. Los modelos finales fueron Random Forest-SMOTE-TPE ($T = 100$, profundidad 8, hoja mínima 5, $\sqrt{p}$; F1 de selección interna = 0.636521), XGBoost-SMOTE-Random ($T = 100$, profundidad 2) y regresión logística-SMOTE-Grid ($C = 0.53367$, penalización L_1). La Tabla 4 muestra su desempeño en el test de 1,409 clientes, que ya había sido expuesto en una entrega anterior y por tanto es evidencia suplementaria.

La prueba de DeLong sobre los scores de los tres modelos fijos, con corrección de Holm para la familia de tres pares, no detectó diferencias de AUC: los tres p ajustados fueron 1.0 y todos los intervalos de la diferencia incluyen el cero. Para Random Forest frente a XGBoost, $\Delta\text{AUC} = 0.0017$ con IC $[-0.0034, 0.0070]$ (p sin ajustar 0.524). Esto describe ausencia de diferencia detectable, no equivalencia.

Hay un intercambio entre detección y precisión: la regresión logística detecta más abandonos (Recall 0.794) a costa de menor Precision (0.515), mientras que Random

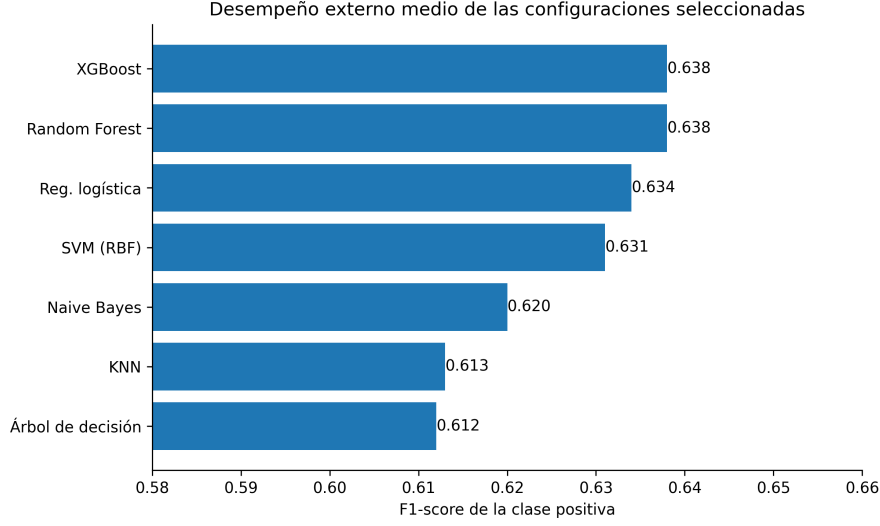


Fig. 3 F1-score medio de las configuraciones seleccionadas para las siete familias de modelos. Random Forest y XGBoost alcanzaron los mayores valores de F1, ambos alrededor de 0.638, seguidos por regresión logística con 0.634.

Table 4 Desempeño de los tres modelos finalistas en el test histórico (1,409 clientes), con la regla de decisión nativa de cada estimador.

Modelo	Accuracy	Precision	Recall	F1	AUC-ROC	Brier	ECE
Random Forest-SMOTE-TPE	0.768	0.547	0.735	0.627	0.844	0.153	0.116
XGBoost-SMOTE-Random	0.769	0.546	0.765	0.637	0.842	0.154	0.113
Reg. logística-SMOTE-Grid	0.747	0.515	0.794	0.625	0.842	0.166	0.142

Table 5 Comparación de AUC-ROC entre los modelos finalistas en el test (DeLong con corrección de Holm; intervalos bootstrap pareados del 95%).

Comparación	ΔAUC	IC 95%	p sin ajustar	p (Holm)
RF-XGBoost	+0.0017	$[-0.0034, 0.0070]$	0.524	1.000
RF-Logística	+0.0014	$[-0.0055, 0.0083]$	0.698	1.000
XGBoost-Logística	-0.0003	$[-0.0073, 0.0066]$	0.921	1.000

Forest y XGBoost tienen mayor Precision (0.547 y 0.546). La Figura 4 resume los valores de AUC-ROC de los tres finalistas; no representa las curvas ROC completas, sino su comparación puntual mediante el área bajo la curva.

Los intervalos bootstrap pareados del 95% de los tres finalistas se solapan ampliamente: F1 de $[0.596, 0.660]$ para Random Forest, $[0.606, 0.667]$ para XGBoost y $[0.597, 0.653]$ para la regresión logística; AUC-ROC de $[0.821, 0.864]$, $[0.820, 0.862]$ y $[0.820, 0.862]$; y Recall de $[0.693, 0.781]$, $[0.725, 0.807]$ y $[0.754, 0.832]$. Son condicionales a los

modelos y no incluyen la incertidumbre de elegir entre 112 configuraciones. La Tabla 5 detalla las comparaciones de AUC.

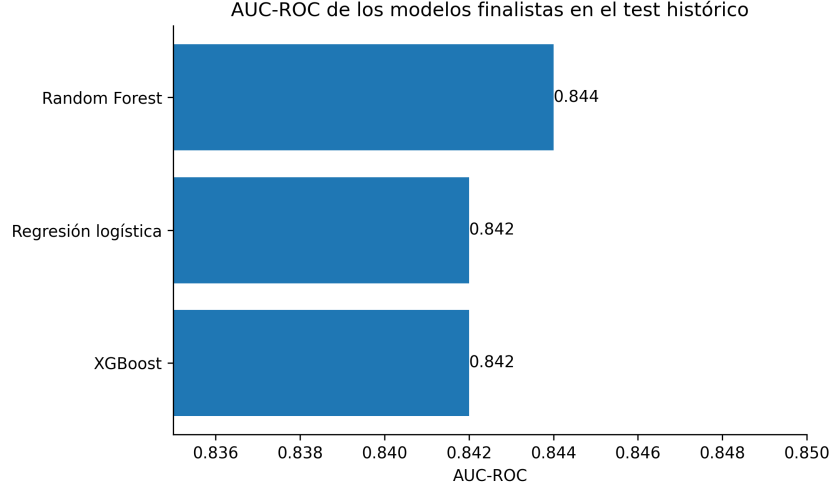


Fig. 4 Comparación del AUC-ROC de los tres modelos finalistas en el test histórico. Random Forest obtuvo un AUC de 0.844, mientras que XGBoost y regresión logística alcanzaron 0.842. Las diferencias fueron pequeñas y no se detectaron diferencias estadísticamente significativas mediante la comparación de AUC.

Optimizadores. A igual presupuesto, TPE, Random y el algoritmo genético mejoraron el F1 interno acumulado frente al orden de Grid solo con presupuestos bajos (a 8 evaluaciones, ganancias de 0.0069, 0.0066 y 0.0063, respectivamente), pero a las 24 evaluaciones las diferencias fueron despreciables (+0.0004, -0.0003 y -0.00005) y las razones de tiempo, cercanas a uno. En KNN, árbol de decisión y Random Forest el espacio discreto de 24 candidatos se agota, por lo que los cuatro optimizadores empatan. No se observó una ventaja universal de calidad ni de velocidad.

Calibración e interpretabilidad. La calibración sigmoide redujo conjuntamente Brier y ECE en 80 de las 96 configuraciones con probabilidades nativas, sin mejoría universal. En el análisis con SHAP [11], las variables de mayor importancia media absoluta fueron `Contract`, `InternetService` y `tenure` en Random Forest (0.077, 0.069 y 0.068) y `Contract`, `tenure` e `InternetService` en XGBoost (0.105, 0.093 y 0.073), como muestra la Figura 5. LIME [12] mostró estabilidad en las variables principales pero fidelidad probabilística variable entre casos. El análisis de casos individuales mostró que los falsos negativos de alta confianza reciben fuertes contribuciones negativas de un contrato de dos años y una antigüedad alta, lo que ilustra una limitación del modelo y no una explicación causal del abandono.

4 Conclusiones

El experimento identificó a Random Forest con SMOTE como uno de los dos modelos con mayor F1 positivo medio (0.638, AUC-ROC de 0.844), junto con XGBoost con

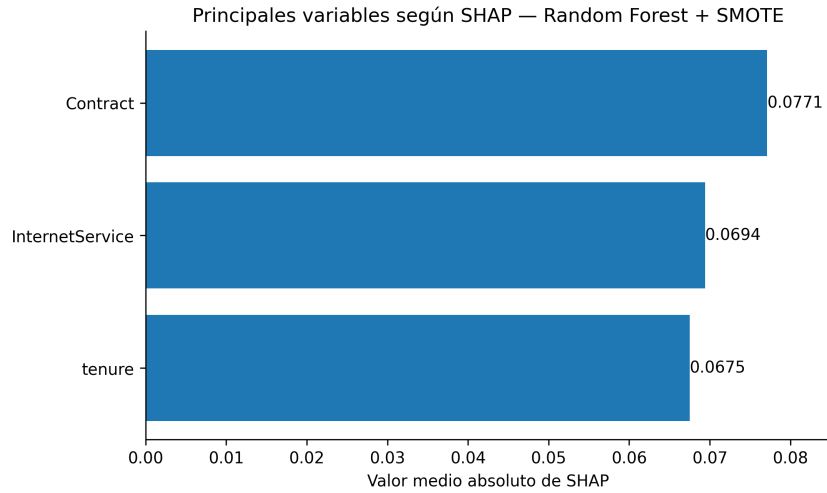


Fig. 5 Importancia global SHAP del modelo Random Forest con SMOTE sobre los 5,634 clientes de desarrollo. Contrato, servicio de internet y antigüedad son las variables con mayor contribución media absoluta entre las variables reportadas.

SMOTE (0.638), pero no permite afirmar que supere a ninguno de los otros finalistas: las diferencias de F1 y de AUC fueron pequeñas y no detectables con los procedimientos usados, y Random Forest fue además la familia de mayor costo de búsqueda, de modo que no mejoró simultáneamente en desempeño y tiempo de cómputo frente a los demás modelos. El resultado más claro es el efecto del balanceo: SMOTE mejoró el F1 en seis de siete familias, un aporte mayor que cambiar de optimizador, cuyas diferencias solo aparecieron con presupuestos de búsqueda bajos. Estos resultados se utilizan como línea base para la etapa posterior de formulación y evaluación del modelo propuesto.

Entre las limitaciones destacan el uso de un único conjunto de datos, de modo que los folds solapados no son bloques independientes y las pruebas de Friedman y Nemenyi son exploratorias; que el test histórico ya estaba expuesto y es evidencia suplementaria; que los intervalos de confianza son condicionales a los modelos y no incluyen la incertidumbre de selección entre 112 configuraciones; y que las variables describen el estado del cliente en un corte y no necesariamente anticipan el abandono futuro. Tampoco se dispone de costes de intervención, por lo que el Recall obtenido, acompañado de falsas alertas, no puede traducirse en beneficio económico. Como trabajo futuro se propone una evaluación prospectiva e independiente con estabilidad temporal, la incorporación de costes explícitos de retención para elegir el umbral de decisión, y la comparación prospectiva con una variante que busque superar simultáneamente a estas referencias en desempeño y tiempo de cómputo.

5 Modelo propuesto: trabajo en curso

Los resultados presentados en la Sección 3 corresponden a la comparación de siete modelos de referencia bajo un protocolo experimental controlado. La siguiente etapa consiste en diseñar y evaluar un modelo propio a partir de los hallazgos

obtenidos en esta comparación. En esta versión del manuscrito, la arquitectura, formulación matemática, estrategia de entrenamiento y justificación del modelo propuesto permanecen pendientes de definición.

6 Configuración experimental del modelo propuesto

7 Resultados del modelo propuesto

Declaraciones

- **Disponibilidad de datos:** el conjunto Telco Customer Churn es público y puede descargarse desde Kaggle (<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>).
- **Disponibilidad de código:** los notebooks de EDA, preprocesamiento y modelamiento avanzado están disponibles en https://github.com/Nataly-Cardenas/Telco_Churn_Entregable
- **Disponibilidad de JBOOK:** los notebooks de EDA, preprocesamiento y modelamiento avanzado están disponibles en https://nataly-cardenas.github.io/Telco_Churn_Entregable/intro.html
- **Contribución de las autoras:** N. Cárdenas y L. Rivera participaron conjuntamente en el ETL, el análisis exploratorio, el modelado y los experimentos, así como en la redacción del manuscrito.
- **Conflicto de intereses:** las autoras declaran no tener conflictos de interés.

References

- [1] Hosmer, D.W., Lemeshow, S., Sturdivant, R.X.: Applied Logistic Regression, 3rd edn. Wiley, Hoboken, NJ (2013)
- [2] Breiman, L.: Random forests. *Machine Learning* **45**(1), 5–32 (2001)
- [3] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794 (2016)
- [4] Cover, T., Hart, P.: Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* **13**(1), 21–27 (1967)
- [5] Domingos, P., Pazzani, M.: On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning* **29**, 103–130 (1997)
- [6] Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: *Classification and Regression Trees*. Wadsworth, Belmont, CA (1984)
- [7] Cortes, C., Vapnik, V.: Support-vector networks. *Machine Learning* **20**(3), 273–297 (1995)

- [8] Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* **16**, 321–357 (2002)
- [9] He, H., Bai, Y., Garcia, E.A., Li, S.: ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In: 2008 IEEE International Joint Conference on Neural Networks, pp. 1322–1328 (2008)
- [10] Varma, S., Simon, R.: Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics* **7**, 91 (2006)
- [11] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. In: *Advances in Neural Information Processing Systems*, vol. 30, pp. 4765–4774 (2017)
- [12] Ribeiro, M.T., Singh, S., Guestrin, C.: “why should I trust you?”: Explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144 (2016)
- [13] IBM: Telco Customer Churn (conjunto de datos de ejemplo de IBM). Kaggle (2018). <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- [14] Cramér, H.: *Mathematical Methods of Statistics*. Princeton University Press, Princeton, NJ (1946)
- [15] Holm, S.: A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* **6**(2), 65–70 (1979)
- [16] Pedregosa, F., *et al.*: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011)
- [17] Lemaître, G., Nogueira, F., Aridas, C.K.: Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research* **18**(17), 1–5 (2017)
- [18] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 2623–2631 (2019)
- [19] Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., Gagné, C.: DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research* **13**, 2171–2175 (2012)
- [20] Friedman, M.: The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association* **32**(200),

675–701 (1937)

- [21] Demšar, J.: Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research* **7**, 1–30 (2006)